

CHAPTER THREE

Mobile Application Design and Implementation

3.1 Introduction

The GENZ Studios project is composed of three integrated components: a network infrastructure layer, an artificial intelligence chatbot server, and a cross-platform mobile application. This chapter focuses exclusively on the mobile application component, which constitutes the primary client-facing interface of the entire system and represents the responsibility of the current author. The network and AI chatbot components are addressed in separate chapters by the respective team members.

The GENZ Studios mobile application is a fully featured studio booking platform designed to connect photography studio clients with studio management staff. It is built with the Flutter framework and targets Android, iOS, and Web platforms from a single unified codebase. The backend is powered entirely by Amazon Web Services through the AWS Amplify v2 Dart SDK, providing authentication, a real-time GraphQL API, a managed NoSQL database, and scalable object storage.

The application serves two distinct user roles: clients, who browse studios, submit booking requests, communicate with support staff, and interact with an integrated AI assistant; and employees, who manage studios, approve or reject bookings, handle support communications, generate analytical reports, and administer user accounts. Role determination is performed automatically at login using Cognito group membership, with each role receiving a tailored interface and a restricted set of permissions.

This chapter is structured as follows. Section 3.2 describes the system architecture. Section 3.3 presents the development methodology and the design patterns applied. Section 3.4 specifies the database design. Section 3.5 details each functional module. Section 3.6 documents the user interface design. Section 3.7 explains the real-time data strategy. Section 3.8 addresses security design. Section 3.9 summarises the performance optimisations applied. Section 3.10 concludes the chapter.

3.2 System Architecture

3.2.1 Architectural Overview

The application adopts a Cloud-First Serverless Architecture that eliminates the need for self-managed servers while providing automatic scaling, high availability, and pay-per-use economics. The architecture is organised into four distinct layers, each with clearly bounded responsibilities, as described in Table 3.1.

Table 3.1 - Architectural Layers

Layer	Technology	Responsibility
Presentation Layer	Flutter (Dart)	All UI rendering, animations, and user interaction across all platforms
State Management Layer	Provider / ChangeNotifier	Bridge between UI and data; holds shared state; triggers selective rebuilds
Service Layer	AWSStorageService (Dart class)	Encapsulates all GraphQL mutations, queries, subscriptions, and S3 operations
Cloud Backend Layer	AWS Amplify v2	Authentication (Cognito), real-time API (AppSync / GraphQL), database (DynamoDB), and object storage (S3)

The AWS infrastructure is deployed in the eu-central-1 region. The Cognito User Pool identifier is eu-central-1_mVZjIjiEM and the AppSync GraphQL endpoint is hosted at a dedicated subdomain under appsync-api.eu-central-1.amazonaws.com. This single-region deployment minimises cross-region latency for the primary target audience while simplifying compliance considerations.

3.2.2 Technology Stack

Each technology in the stack was selected based on a formal evaluation of performance, developer productivity, ecosystem maturity, and alignment with the project requirements, as summarised in Table 3.2.

Table 3.2 - Technology Stack Justification

Technology	Version / Service	Justification

Flutter	Stable Channel	Single codebase targets Android, iOS, and Web; GPU-accelerated rendering; rich built-in widget library
Dart	Latest Stable	Strongly typed, AOT-compiled language; null safety enforced at compile time
AWS Amplify Dart SDK	v2	Unified Dart library integrating Cognito, AppSync, and S3 with a type-safe GraphQL client
AWS Cognito	Managed Service	Industry-standard user pool with JWT tokens, group-based authorisation, and password-policy enforcement
AWS AppSync	GraphQL Managed	Real-time GraphQL subscriptions, built-in optimistic concurrency control, and owner-based auth directives
AWS DynamoDB	Managed NoSQL	Single-digit millisecond latency; automatic scaling; Global Secondary Indexes generated from the Amplify schema
AWS S3	Managed Object Store	Durable scalable object storage for studio and profile images with time-limited pre-signed URLs
Provider	^6.x	Officially recommended Flutter state management; minimal boilerplate; selective widget rebuild via context.watch
SharedPreferences	Stable	Persistent local key-value store for onboarding flags, chatbot session ID, language, and theme preference
flutter_localizations	Flutter SDK	Full Arabic RTL and English LTR localisation with 200+ internationalised strings
pdf + printing	Stable	On-device PDF generation and printing or sharing of booking confirmations and reports

image_picker	Stable	Cross-platform photo selection from camera roll on mobile and file dialog on Web
--------------	--------	--

3.3 Development Methodology and Design Patterns

3.3.1 Software Development Model

The project was developed following an Iterative and Incremental Development (IID) approach. Unlike the traditional Waterfall model, which requires each phase to be completed in full before proceeding, IID allows requirements to be refined continuously as the system grows and new feedback is incorporated. Each iteration of development covered five phases in sequence.

The Planning phase identified the features and tasks allocated to the iteration. The Design phase produced data model diagrams, screen wireframes, and service interface contracts. The Implementation phase produced working code that was immediately integrated into the main codebase. The Testing phase included unit tests for individual functions and manual integration tests for complete user workflows. The Review phase evaluated the output against the requirements, yielding adjustments that fed directly into the next iteration's planning phase.

This cyclical approach was especially valuable for a mobile application project because the user interface requirements evolved significantly as early screens revealed usability issues that were not apparent in the wireframe stage. The ability to incorporate feedback and revise designs within the same overarching development cycle reduced the cost of change and produced a more refined final product.

3.3.2 Design Patterns Applied

Provider Pattern (State Management)

The Provider Pattern is the central mechanism for managing shared application state. A single `AppState` class extends Flutter's `ChangeNotifier` and holds all data that must be shared across multiple screens: the authenticated user's profile, the list of studios, all booking records, and all notifications. Widgets call `context.watch<AppState>()` to subscribe to changes, and are automatically rebuilt whenever `notifyListeners()` is called. This eliminates the need for manual `setState()` calls scattered across the widget tree and ensures the user interface is always consistent with the underlying data.

The AppState class also manages all GraphQL subscription lifecycles. It exposes dedicated methods such as startStudiosSubscription(), startBookingsSubscription(), and startNotificationsSubscription() that are called once after login and automatically clean up their StreamSubscription objects when dispose() or clearUser() is invoked. This guarantees that no stale subscriptions leak memory after the user signs out.

Repository Pattern (Service Layer)

The Repository Pattern is applied through the AWSStorageService class, which acts as the sole data repository for the entire application. All GraphQL mutations, queries, and subscriptions; all S3 upload, download, and delete operations; and all authentication calls are encapsulated within this single class using static methods. No screen or widget in the application communicates directly with the AWS SDK; every data operation goes through AWSStorageService. This strict separation of concerns makes each layer independently testable and ensures that any change to the backend API requires modification in only one location.

Singleton Pattern (Shared User State)

A Singleton Pattern is applied within AWSStorageService through a static map named currentUser, which holds the authenticated user's email, name, type, and profile image path. Because it is a class-level static member, it is guaranteed to be a single instance shared across all call sites without requiring a dependency injection mechanism. This prevents redundant network requests to fetch the profile on every screen transition and provides a consistent view of the user's identity throughout the application lifecycle.

3.4 Database Design

3.4.1 Database Technology Selection

AWS DynamoDB was selected as the primary database for the GENZ Studios application. DynamoDB is a fully managed serverless NoSQL database that provides consistent single-digit millisecond read and write latency at any scale without capacity planning. The data access patterns of the application, which centre on fetching a studio list, querying bookings by client email, and retrieving notifications by recipient, are all well served by DynamoDB's key-value and document data model.

All data models are defined using the AWS Amplify GraphQL schema language. Amplify automatically provisions the corresponding DynamoDB tables, Global Secondary Indexes (GSIs), and AppSync resolvers from this schema declaration, ensuring that the database structure always remains in sync with the application models. Owner-based and group-based

access control policies are also declared within the schema using the `@auth` directive and are enforced by AppSync at the API layer before any data reaches the application.

3.4.2 Data Entity Specifications

The system defines five core data entities. Each entity is described in full below.

Entity 1: UserProfile

The UserProfile entity stores the profile data for every registered user, regardless of their role. It is created automatically on first login if no record exists and is updated whenever the user modifies their name or profile photo.

Table 3.3 - UserProfile Entity Schema

Field	GraphQL Type	Constraints	Description
id	ID	Primary Key, Auto-generated	Universally unique identifier (UUID)
email	String	Required, GSI Partition Key	Cognito email; owner field; used as the lookup key
name	String	Optional	Display name shown across the application
image	String	Optional	S3 object key of the profile photo
type	String	Optional	User role: client or employee
chatEnabled	Boolean	Default: true	Employee-controlled flag to enable or disable client chat
lastUpdated	AWSDateTime	Required	ISO 8601 timestamp of the last modification

Entity 2: Studio

The Studio entity represents a photography studio that clients can browse and book. Studios are created and managed exclusively by employees. Each studio belongs to one of five categories that carry distinct icons and accent colours in the UI: Portrait, Product, Wedding, Video, and Fashion.

Table 3.4 - Studio Entity Schema

Field	GraphQL Type	Constraints	Description
id	ID	Primary Key, Auto-generated	Universally unique identifier
name	String	Required	Display name of the studio
type	String	Required	Category: Portrait, Product, Wedding, Video, or Fashion
pricePerHour	Int	Required	Hourly rental rate in US dollars
description	String	Optional	Detailed description of the studio and its facilities
image	String	Optional	Pipe-delimited () S3 object keys for multiple studio images
available	Boolean	Default: true	Whether the studio is open for new bookings

Entity 3: BookingRequest

The BookingRequest entity records every booking request submitted by a client. The clientEmail field is designated as the owner field for row-level security. The fullStartDateTime and fullEndDateTime fields store ISO 8601 timestamps that are used by the atomic conflict detection algorithm to determine time overlaps precisely.

Table 3.5 - BookingRequest Entity Schema

Field	GraphQL Type	Constraints	Description
id	ID	Primary Key, Auto-generated	Universally unique identifier
clientEmail	String	Required, Owner Field, GSI	Authenticated client email for row-level security

clientName	String	Optional	Full name of the client
clientPhone	String	Optional	Contact phone number of the client
studio	String	Required	Name of the selected studio
date	String	Required	Human-readable date range string for display
hours	String	Required	Human-readable time range string for display
fullStartDateTime	AWSDateTime	Required	ISO 8601 start timestamp for conflict detection
fullEndDateTime	AWSDateTime	Required	ISO 8601 end timestamp for conflict detection
price	String	Required	Total calculated booking price in USD
equipment	String	Optional	Comma-separated list of additional equipment add-ons
status	String	Default: Pending	Booking lifecycle state: Pending, Approved, Rejected, or Cancelled

Entity 4: ChatMessage

The ChatMessage entity stores all messages exchanged between clients and support staff, as well as formal support tickets. The messageType field distinguishes between regular chat messages and structured support tickets, allowing the UI to route each type to its own dedicated screen.

Table 3.6 - ChatMessage Entity Schema

Field	GraphQL Type	Constraints	Description
-------	--------------	-------------	-------------

id	ID	Primary Key, Auto-generated	Universally unique identifier
clientEmail	String	Required, Owner Field, GSI	Email of the client in the conversation thread
senderEmail	String	Optional	Email of the message sender (client or employee)
senderName	String	Optional	Display name of the sender
text	String	Optional	Full text body of the message
time	AWSDatetime	Required	ISO 8601 timestamp when the message was created
messageType	String	Default: chat	Message context: chat for support chat, ticket for support tickets
parentId	String	Optional	Reference to the parent message ID for threaded ticket replies

Entity 5: AppNotification

The AppNotification entity manages in-app notifications delivered to both clients and employees. Client notifications are addressed directly to the client's email. Employee notifications are routed to the sentinel key `__employees__`, enabling all staff members to share a single notification inbox without requiring knowledge of individual employee email addresses. A legacy key `EMPLOYEE_INBOX` is also maintained for backward compatibility.

Table 3.7 - AppNotification Entity Schema

Field	GraphQL Type	Constraints	Description
id	ID	Primary Key, Auto-generated	Universally unique identifier
clientEmail	String	Required, Owner Field, GSI	Recipient identifier: client email or <code>__employees__</code> for all staff

title	String	Optional	Short notification headline shown in bold
body	String	Optional	Full notification message body
type	String	Optional	Notification category: Approved, Rejected, new_booking, info, or chat_opened
time	AWSDateTime	Optional	ISO 8601 timestamp when the notification was created
read	Boolean	Default: false	Whether the notification has been read by the recipient

3.4.3 Access Control Model

Access control is enforced at the GraphQL API layer using AppSync's `@auth` directive with two complementary strategies. The owner strategy designates `clientEmail` as the owner field on `BookingRequest`, `ChatMessage`, and `AppNotification`. AppSync automatically injects a filter ensuring that clients can only read and write records where their Cognito-authenticated email matches the `clientEmail` value. No application-level filtering is required or trusted for this check.

The group strategy is applied to the employee role. Users whose Cognito account belongs to the Employee group are granted full read and write access across all records of all entities, overriding the owner restriction. This two-tier model means that an employee can query any booking for management purposes while a client is strictly limited to their own records. The authenticated user's email is always retrieved directly from the Cognito JWT token via `fetchUserAttributes()` and is never read from a local cache, preventing spoofing through stale values.

3.4.4 Optimistic Concurrency Control

AppSync's built-in optimistic concurrency control is enabled on all DynamoDB tables through the `_version` field appended to every record by the Amplify DataStore conflict resolution layer. Every mutation that modifies an existing record must supply the current version number, which is fetched immediately before the mutation is sent. If two clients attempt to modify the same record simultaneously, the second mutation arrives with an outdated version

number, and AppSync rejects it with a version conflict error. This prevents silent data corruption under concurrent access.

3.5 System Modules

3.5.1 Authentication and Session Module

The authentication module manages user identity throughout the application lifecycle and is implemented across four screens. The module is powered by AWS Cognito and integrates with the Amplify Auth category.

The Welcome Screen (WelcomeScreen) is the entry point for unauthenticated users. It presents a Sign In action for existing users and a Register action for new users, displayed over the animated GENZ Studios logo.

The Registration Screen (RegisterScreen) collects the user's full name, email address, and password. Password validation is enforced client-side before submission: the password must be at least eight characters long and must contain at least one uppercase letter, one lowercase letter, one digit, and one special character. These requirements mirror the Cognito User Pool password policy. On successful registration, Cognito sends a one-time password (OTP) to the provided email address and the user is navigated to the OTP confirmation screen.

The OTP Confirmation Screen (ConfirmSignUpScreen) displays a six-digit code entry field. When the user submits the correct code, the Cognito account is confirmed and the user is automatically signed in. A resend code option is provided for cases where the email is not received within the expected window.

The Sign In Screen (TestScreen) accepts the user's email and password and calls `Amplify.Auth.signIn()`. On success, the module fetches the Cognito JWT token, reads the user's group membership to determine their role, loads or creates the `UserProfile` record from DynamoDB, and routes the user to the appropriate screen: `EmployeesScreen` for employees or `ClientScreen` for clients.

The Forgot Password flow is a two-step process. In step one, the user enters their email address and a reset code is sent to that address by Cognito. In step two, the user enters the received code together with a new password that meets the same password policy requirements. On success, the user is returned to the sign in screen.

The SplashScreen executes on every cold launch. It calls `fetchAuthSession()` to detect an existing valid session. If a session is found, the module skips the authentication screens entirely, loads the user profile, and navigates directly to the main application screen. This

provides a seamless experience for returning users. Portrait orientation is locked on mobile devices at this point using `SystemChrome.setPreferredOrientations()`, and the global text scale factor is clamped to the range 0.85 to 1.20 to protect layout integrity across all accessibility font size settings.

3.5.2 Onboarding Module

The onboarding module presents a guided interactive tour to first-time clients. The tour is displayed automatically on the first launch of the `ClientScreen` and is skipped on all subsequent launches by reading an `onboarding_done` boolean flag from `SharedPreferences`. Users who wish to revisit the tour may do so at any time through the App Tour option in the navigation drawer.

The tour consists of five pages delivered through a horizontally scrollable `PageView` widget with smooth fade transitions and animated dot indicators at the bottom. Table 3.8 describes the content of each page.

Table 3.8 - Onboarding Tour Pages

Page	Title	Content Summary
1	Welcome to GENZ Studios	Splash introduction showing the app logo and a brief mission statement
2	Browse Studios	Four-step guide explaining how to browse the studio grid, view studio details, check availability, and understand category icons
3	Book a Studio	Four-step guide covering studio selection, filling in the booking form, selecting equipment add-ons, and submitting the request
4	Track Your Bookings	Four-step guide explaining how to find bookings in My Bookings, read status badges (Pending, Approved, Rejected), and cancel a pending booking
5	Support and Help	Four-step guide introducing the support chat, the ticketing system, the AI chatbot assistant, and the notification inbox

A Skip button on every page and a Get Started button on the final page both write the `onboarding_done` flag and dismiss the tour. The onboarding screens use animated fades and icon transitions to maintain visual engagement without distracting from the instructional content.

3.5.3 Studio Management Module

The studio management module provides employees with full CRUD (Create, Read, Update, Delete) control over studio listings and presents available studios to clients in a browsable grid interface.

Employees create a studio by entering its name, selecting a category from five options (Portrait, Product, Wedding, Video, or Fashion), specifying the hourly price in USD, providing a description, setting the initial availability status, and uploading one or more photographs. Uploaded images are sent to AWS S3 under the path `public/studios/{userEmail}-{timestamp}.{extension}`. The resulting S3 object keys are joined with the pipe character (`|`) and stored in a single String field, allowing multiple images per studio without requiring a separate image table.

Studio records are fetched using a raw GraphQL `listStudios` query. The returned S3 object keys are then converted to pre-signed HTTPS URLs with a one-hour expiry by calling `Amplify.Storage.getUrl()`. Updates to existing studios retrieve the current `_version` field before issuing the mutation, satisfying AppSync's optimistic concurrency requirement. When a studio is deleted, all associated S3 images are removed first to prevent orphaned objects in the bucket.

The module subscribes to three AppSync events: `onCreate`, `onUpdate`, and `onDelete`. When any of these events fires, all connected client devices receive the update in real time without a page refresh. A polling timer running every 90 seconds provides a fallback in case of subscription disconnection.

3.5.4 Booking Management Module

The booking management module handles the full lifecycle of a booking request, from submission through to completion. It implements an atomic double-check algorithm to eliminate the possibility of double-bookings on the same studio.

Booking Submission Form

The booking form is presented within the Booking View of the ClientScreen. The client selects a studio from a dropdown populated from the live studio list, enters their first name, last name, and phone number, picks a start date from a calendar date picker, selects a start

hour from a dropdown ranging from 6:00 AM to 10:00 PM, selects an end date and an end hour using the same controls, optionally adds equipment from the catalogue shown in Table 3.9, and accepts the terms and conditions via a mandatory checkbox before submitting.

Table 3.9 - Available Equipment Add-Ons and Pricing

Equipment Item	Additional Cost (USD)
Pro Lighting Kit	\$50 per booking
4K Camera	\$100 per booking
Reflector Set	\$20 per booking
Smoke Machine	\$40 per booking
Background Stands	\$30 per booking

The total price is recalculated in real time whenever the studio selection, start date, start hour, end date, end hour, or equipment selection changes. The formula is: $\text{Total} = (\text{hourlyRate} * \text{numberOfHours}) + \text{sum of selected equipment costs}$.

Atomic Conflict Detection Algorithm

To prevent double-bookings, the system implements a three-step atomic procedure within `AWSSStorageService.saveBookingAtomic()`.

In step one (pre-check), a GraphQL query retrieves all non-rejected bookings for the selected studio. For each existing booking, the algorithm checks for time overlap using the condition: a conflict exists if the new booking's start time is before the existing booking's end time AND the new booking's end time is after the existing booking's start time. If any conflict is detected, the error code `studio_booked` is returned immediately and no record is written.

In step two (save), if no conflict is found, the new `BookingRequest` record is written to DynamoDB via an AppSync mutation. Notifications are dispatched simultaneously: one to the employee inbox (addressed to `__employees__`) informing staff of a new booking request, and one to the client confirming that their request has been received.

In step three (post-check), a second conflict query is executed, this time excluding the newly created record by its ID. This detects any race condition that may have occurred between the pre-check and the save when two clients submit overlapping bookings within the same millisecond window. If a conflict is detected in the post-check, the newly created record is immediately deleted and the error code `studio_booked` is returned.

Booking Lifecycle

A booking progresses through a defined sequence of status values. It is created with status `Pending`. An employee may change it to `Approved` or `Rejected` from the Booking Detail

Screen. A client may change a Pending booking to Cancelled from the My Bookings Screen. Each status transition triggers a notification to the affected party.

3.5.5 Real-Time Chat and Support Ticketing Module

The chat and support module provides two distinct communication channels: a real-time support chat between clients and employees, and a structured ticketing system for formal issue reporting.

Live Support Chat

The ChatScreen presents a standard messaging interface. Before loading messages, the screen reads the chatEnabled field from the client's UserProfile. If chat is disabled by an employee, the input field is hidden and a visual lock indicator with an explanatory message is displayed in its place. When chat is enabled, messages are presented in a bubble layout: client messages appear on the right with a filled background and employee messages appear on the left with a contrasting style, each showing the sender's display name and the formatted timestamp.

New messages arrive via an AppSync onCreate subscription, delivering real-time updates to both the client and the employee without any polling overhead. Employees can toggle the chatEnabled flag for any client directly from the Clients section of the Employee Dashboard. The toggle updates the UserProfile record in DynamoDB and the change is instantly reflected on the client's device through a dedicated UserProfile subscription running on the ChatScreen.

Support Ticketing System

The NewTicketScreen presents a structured form for submitting a support ticket. The client provides a subject line, a message body, and selects a category from four options: Payment, Booking, Technical, and Other. The submitted ticket is stored as a ChatMessage record with the messageType field set to ticket and a notification is sent to the employee inbox.

The MyTicketsScreen groups all messages with messageType equal to ticket into threaded conversations, displaying the subject, category with colour coding, the latest reply excerpt, the reply count, and the time elapsed since the last activity. Employees reply through the ChatScreen, and the MyTicketsScreen polls for updates every five seconds to capture new replies promptly.

3.5.6 Notification Module

The notification module keeps both clients and employees informed of relevant system events through an in-app notification inbox. Table 3.10 documents all supported notification events, their recipients, and the type codes used for icon and style selection.

Table 3.10 - Notification Event Matrix

Triggering Event	Recipient	Type Code	Title
Client submits new booking	All employees (__employees__)	new_booking	New Booking Request
Employee approves booking	Client	Approved	Your Booking is Confirmed
Employee rejects booking	Client	Rejected	Booking Update
Booking confirmed to submitter	Client	info	Booking Submitted
Employee enables client chat	Client	chat_opened	Support Chat Opened

Notifications are fetched for clients by querying records where clientEmail matches the authenticated user's email, and for employees by querying where clientEmail equals __employees__. Since AppSync owner-auth subscriptions require server-side filter arguments not available for this entity type, notifications are refreshed by a polling timer running every 20 seconds.

The NotificationsScreen displays all notifications in reverse chronological order, grouped by date (Today, Yesterday, and then the explicit date for older items). Each card shows an icon corresponding to the type code, the title, the body text, and the relative time elapsed. Unread notifications are visually distinguished. Tapping a notification marks it as read by updating the read field in DynamoDB, which decrements the unread badge counter on the navigation element in real time.

When a new notification arrives while the user is actively using the application, a custom animated banner slides in at the bottom of the current screen. The banner displays the notification title and body with an icon, and includes a View button that navigates directly to the notifications screen.

3.5.7 Image Storage Module

The image storage module handles the upload, retrieval, and deletion of all binary assets within the application, including studio photographs and user profile pictures. All images are stored in a dedicated AWS S3 bucket under the eu-central-1 region.

Studio images are stored under the S3 path `public/studios/{userEmail}-{timestamp}.{extension}` and profile photos under `public/profile-images/{userEmail}-{timestamp}.{extension}`. On platforms that provide a file

path (Android, iOS, macOS), `Amplify.Storage.uploadFile()` is used with the local path. On Web, where a file path is unavailable, `Amplify.Storage.uploadData()` is used with the raw binary bytes obtained from the browser's file API. This dual-path approach ensures complete cross-platform compatibility from a single implementation.

Pre-signed HTTPS URLs are generated on demand by calling `Amplify.Storage.getUrl()` with a one-hour expiry setting. This expiry balances security (limiting the window during which a leaked URL is valid) against performance (avoiding the need to regenerate URLs on every render). When a studio or profile is deleted, all associated S3 objects are explicitly removed using `Amplify.Storage.remove()` before the DynamoDB record is deleted, preventing orphaned objects from accumulating in the bucket.

3.5.8 AI Chatbot Integration Module

The AI chatbot integration module embeds the project's artificial intelligence assistant directly within the mobile application. The chatbot is accessible from any screen in the application via a persistent floating action button that navigates to the dedicated `ChatbotScreen`.

The module communicates with the AI server via standard HTTP over the network using two REST endpoints. The primary endpoint accepts POST requests at the `/chat` path and expects a JSON body containing the user's message text and a session identifier. The server processes the request and returns a JSON response containing the chatbot's reply under the key `reply`. The reset endpoint accepts POST requests at the `/reset` path and instructs the server to clear the conversation history for the provided session, effectively starting a fresh conversation.

Network timeouts are set to 30 seconds for chat requests and 10 seconds for reset requests. The `AppConfig` class centralises these values alongside the server base URL, the S3 URL expiry duration of one hour, and feature flags for enabling or disabling the chatbot and notification sounds. A configuration validation check at startup logs a warning if the configured URL contains a local or private IP address, which would indicate a development-only setting deployed to production.

On first launch of the `ChatbotScreen`, a unique session identifier is generated from the current Unix timestamp in the format `sess_{timestamp}` and persisted in `SharedPreferences`. This identifier is sent with every request so that the server can maintain conversation context across multiple exchanges within the same session. Conversation history is also saved locally as a JSON list in `SharedPreferences`, allowing the client to scroll back through previous messages after an application restart.

The interface renders client messages in dark-background bubbles aligned to the right and server responses in light-background bubbles aligned to the left. A loading indicator is displayed while the server is processing the request. A set of quick suggestion buttons is

shown initially to help new users begin a conversation. A Clear Chat button invokes the `/reset` endpoint and empties the local history. The screen supports both English and Arabic through the application's full localisation system.

3.5.9 Reporting and Analytics Module

The reporting module provides employees with detailed financial and operational analytics presented in the dedicated ReportsScreen. All booking data is fetched from DynamoDB and aggregated entirely on the client device, avoiding the need for a separate analytics backend.

Bookings are organised into a hierarchical three-level structure: year at the top level, month at the second level, and individual booking records at the third level. Each level is rendered as a collapsible section. For each year and for each month within a year, the module computes the total number of bookings, the count of approved bookings, the count of pending bookings, the count of rejected bookings, the total revenue generated from approved bookings in USD, and the total number of hours booked. A bar chart at the top of the screen visualises monthly booking volumes for quick trend identification.

A ranking table within the employee dashboard lists studios sorted by the number of bookings they have received, providing visibility into which studios are most in demand. An Export PDF button at the top of the ReportsScreen generates a complete formatted report using the pdf and printing packages. The generated document uses the Cairo font to support both Arabic and English content, and can be shared via the device's native share sheet, emailed, or printed directly from the device.

3.5.10 Settings Module

The settings module allows users to personalise the application's language and visual theme. Two language options are supported: English with a left-to-right layout and Arabic with a full right-to-left layout and mirrored navigation elements. Three theme options are supported: Light, Dark, and System Default, where the last option mirrors the operating system's current appearance setting.

Both the language preference and the theme preference are persisted in SharedPreferences and exposed as ValueListenable objects. The root MaterialApp widget is wrapped in nested ValueListenableBuilder widgets subscribed to both preferences, so any change in the SettingsScreen takes effect immediately and globally across the entire application without requiring a restart. The localisation system covers more than 200 user-visible strings, all declared in the AppLocalizations delegate.

3.6 User Interface Design

3.6.1 Design Principles

The GENZ Studios user interface adheres to Material Design 3 guidelines to ensure consistency, accessibility, and platform-native familiarity across Android, iOS, and Web. A custom AppTheme class defines the application's visual identity through two complete theme configurations: a light theme and a dark theme. Both themes are built exclusively from the following colour palette used for interactive elements and status indicators: primary purple (#6C63FF), success green (#22C55E), error red (#EF4444), warning amber (#F59E0B), and gradient end purple (#9B59B6). Backgrounds and surface colours are pure white in the light theme and deep grey in the dark theme.

Shared widget components defined in `app_theme.dart` include `GenzButton` for primary call-to-action buttons, `GenzTextField` for styled input fields, `GenzLogo` for the animated brand mark, and `SectionHeader` for section dividers throughout the dashboard. Using these shared components ensures visual consistency without duplicating style definitions across screens.

3.6.2 Responsive Layout System

The application implements a four-tier responsive layout system using a dedicated Responsive utility class. Every screen queries the current device width at build time and selects the appropriate layout tier, as documented in Table 3.11.

Table 3.11 - Responsive Design Breakpoints

Tier	Width Range	Navigation Element	Studio Grid Columns
Mobile	Below 600 px	Sliding Drawer opened by hamburger icon in the AppBar	1 column
Tablet	600 px to 1024 px	Persistent icon rail sidebar, 68 px wide	2 columns
Desktop	1024 px to 1439 px	Full labelled sidebar, 220 px wide	3 columns
Large Desktop	1440 px and above	Full labelled sidebar, 220 px wide, wider content area	3+ columns

The `Responsive` class exposes static boolean properties (`isMobile`, `isTablet`, `isDesktop`, `isLargeDesktop`) and a convenience factory method that accepts the `BuildContext` and returns

the current breakpoint. Screens use these properties in their build methods to select the correct layout, spacing values, and font size multipliers without conditional nesting.

3.6.3 Application Screens

The application consists of eighteen screens. Table 3.12 provides a complete inventory, and the following subsections describe each screen in detail.

Table 3.12 - Complete Screen Inventory

Screen Name	Flutter Class	Role	Primary Purpose
Splash	SplashScreen	All	Session detection and role-based routing on cold launch
Welcome	WelcomeScreen	All	Authentication entry point with animated logo
Register	RegisterScreen	All	New account creation with password policy validation
OTP Confirm	ConfirmSignUpScreen	All	Six-digit OTP verification after registration
Sign In	TestScreen	All	Email and password authentication
Forgot Password	ForgotPasswordScreen	All	Two-step password reset via email OTP
Onboarding	OnboardingScreen	Client	Five-page guided tour for first-time users
Client Home	ClientScreen	Client	Studio grid, booking form, and primary navigation hub
Studio Detail	StudioDetailScreen	Client	Swipeable photo gallery and studio specifications

My Bookings	MyBookingsScreen	Client	Tabbed view of all personal booking requests
My Tickets	MyTicketsScreen	Client	Threaded list of all submitted support tickets
New Ticket	NewTicketScreen	Client	Structured form for submitting a support ticket
AI Chatbot	ChatbotScreen	Client	Conversational AI assistant via remote HTTP API
Chat	ChatScreen	Both	Real-time support messaging between client and employee
Notifications	NotificationsScreen	Both	Date-grouped notification inbox with read/unread status
Profile	ProfileScreen	Both	User summary with quick links to key sections
Edit Profile	EditProfileScreen	Both	Name and photo update with S3 image upload
Employee Dashboard	EmployeesScreen	Employee	Central management console for all operations
Booking Detail	BookingDetailScreen	Employee	Full booking record with approve, reject, PDF export
Reports	ReportsScreen	Employee	Hierarchical revenue and booking analytics with PDF export
Settings	SettingsScreen	Both	Language and theme preferences
Privacy Policy	PrivacyPolicyScreen	Both	In-app privacy policy text display

Client Home Screen (ClientScreen)

The ClientScreen is the primary interface for clients and serves as the navigation hub for all client-facing functionality. It consists of two views, toggled by the navigation element. The Home View displays a personalised welcome banner containing the client's name against a gradient background, followed by a responsive grid of studio cards. Each card shows the studio's cover photograph at a 16:9 aspect ratio, a category-specific icon and accent border, the studio name, type label, and hourly price displayed as an overlay badge. When a studio has more than one photograph, an additional photo-count badge is shown. Studios with the available flag set to false display a CLOSED overlay badge. A Book button on each card opens the Booking View with that studio pre-selected, and tapping the card body navigates to the Studio Detail Screen.

The Booking View provides the complete booking submission form described in Section 3.5.4. Navigation between the two views is provided by a bottom navigation bar on mobile, an icon rail on tablet, and a full sidebar on desktop. A floating action button in the bottom-right corner is always visible and opens the ChatbotScreen. Studios are refreshed by a polling timer every 90 seconds as a fallback for subscription gaps, bookings are polled every 12 seconds, and notifications are polled every 20 seconds.

Employee Dashboard (EmployeesScreen)

The EmployeesScreen is a multi-section management console accessible exclusively to Cognito Employee group members. It is organised into the following sections, navigable via a labelled sidebar on desktop or a navigation rail on tablet.

The Dashboard section displays key performance indicators: total bookings, approved bookings, pending bookings, rejected bookings, total revenue in USD, and total hours booked. Each KPI has an associated date-range picker allowing the employee to filter metrics to a specific day, week, or month. A bar chart visualises monthly booking volumes. A ranking table lists studios by the number of bookings received. Bookings data is refreshed by a polling timer every 8 seconds.

The Studios section provides full CRUD management of the studio catalogue. Employees can add, edit, and delete studios, toggle their availability, and upload or replace images. The Bookings section lists all booking requests across all clients with filter tabs for All, Pending, Approved, and Rejected. Each card navigates to the BookingDetailScreen.

The Clients section lists all clients who have sent at least one chat message. Each row shows the client's name, email, and chat status (enabled or disabled). Tapping a client opens their ChatScreen thread. An inline toggle switches the chatEnabled flag and the new value is cached locally with a timestamp to avoid redundant GraphQL calls within a five-minute window. Chat messages from all clients are polled every 10 seconds.

The Notifications section shows the shared employee inbox. Notifications are loaded via the

EMPLOYEE_INBOX key and displayed identically to the client notification screen. Employee notifications are polled via the `_listenToEmployeeNotifications` method which runs on screen load and connects the AppState subscription.

Booking Detail Screen (BookingDetailScreen)

The BookingDetailScreen displays the complete record of a single booking request. It shows the client's full name, email, phone number, selected studio, date range, time range, equipment add-ons, total price, and the current status with a colour-coded badge. If the booking is in Pending status, two action buttons are displayed: Approve and Reject. Tapping either button calls `AWSSStorageService.updateBookingStatus()` and dispatches a status notification to the client.

A Print or Export PDF button generates a formatted booking confirmation document on the device using the pdf and printing packages. The document uses the Cairo font, which supports both Arabic and English characters, ensuring that client names and studio names in either language are rendered correctly. The document is opened in the system's native print or share interface. An `_isPrinting` flag prevents duplicate PDF generation if the user taps the button repeatedly while the document is being compiled.

Reports Screen (ReportsScreen)

The ReportsScreen aggregates all booking data into a hierarchical year-month-day structure. Each year is rendered as a collapsible card showing aggregate statistics: total bookings, approved, pending, rejected, total revenue in USD, and total hours booked. Expanding a year reveals a month-level breakdown with the same statistics. Expanding a month reveals individual booking cards grouped by day. Multiple years can be expanded simultaneously for year-over-year comparison. An Export PDF button at the top generates a complete formatted report using the pdf and printing packages, shareable or printable from the device.

AI Chatbot Screen (ChatbotScreen)

The ChatbotScreen presents a conversational interface connected to the project's AI server. Quick suggestion buttons displayed on an empty history help first-time users start a conversation without needing to know what to ask. Messages are submitted via a text field at the bottom of the screen. Each submission sends an HTTP POST to the `/chat` endpoint with the message and session ID, shows a loading indicator, and appends the server's reply to the local message list. A Clear Chat option sends a POST to the `/reset` endpoint, erases the local history from SharedPreferences, and removes the session ID so that a new session is generated on the next message.

3.7 Real-Time Data Strategy

A key architectural decision was the selection of an appropriate real-time update mechanism for each entity type. AWS AppSync supports GraphQL subscriptions over WebSocket, which deliver changes instantly but are subject to limitations in owner-auth configurations. Polling provides a reliable universal fallback but introduces latency proportional to the poll interval and consumes additional network bandwidth. Table 3.13 documents the strategy applied to each entity and the technical justification for each choice.

Table 3.13 - Real-Time Update Strategy

Entity	Strategy	Interval / Events	Justification
Studios	Subscription + Polling fallback	onCreate, onUpdate, onDelete + every 90 s	Studio changes are infrequent; subscriptions cover instant updates; 90 s polling catches missed events after reconnect
Bookings (Employee view)	Subscription only	onCreate, onUpdate, onDelete	Employees have full table access; model subscriptions function without client-side owner filtering
Bookings (Client view)	Polling only	Every 12 seconds	AppSync owner-auth subscriptions require server-side filter arguments not yet supported by the Amplify Dart model subscriptions
Chat Messages	Subscription only	onCreate	Low-latency messaging is critical; subscriptions provide instant delivery without polling overhead
Chat Status (UserProfile)	Subscription only	onCreate + onUpdate	Client must receive immediate feedback when an employee toggles their chat access flag

Notifications	Polling only	Every 20 seconds	AppSync rejects subscriptions on owner-auth types without an authenticated filter argument; polling is sufficient given the non-critical latency requirement
Support Tickets	Polling only	Every 5 seconds	Ticket replies are modelled as ChatMessages; polling provides acceptable freshness for this asynchronous channel
Employee Chat Messages	Polling only	Every 10 seconds	Employee side of chat monitored by periodic refresh to catch messages from all clients without per-client subscriptions

3.8 Security Design

3.8.1 Authentication Security

All API requests to AppSync require a valid Cognito-issued JWT bearer token. The Amplify SDK automatically attaches the token to every request and handles token refresh transparently before expiry. The user's verified email address is always retrieved directly from the Cognito token attributes via `fetchUserAttributes()` and is never read from a local cache, `SharedPreferences` value, or user-supplied input. This prevents a class of authorisation bypass vulnerabilities where a stale or manipulated cached email could match a different user's owner field in DynamoDB.

Password security is enforced by the Cognito User Pool password policy, which requires a minimum of eight characters including at least one uppercase letter, one lowercase letter, one digit, and one special character. These requirements are also validated client-side in the `RegisterScreen` before the registration request is sent, providing immediate feedback without a server round-trip.

3.8.2 Authorisation Security

Row-level data security is enforced at the GraphQL API layer by AppSync's `@auth` directive. The owner strategy ensures that a client's requests are filtered by the server before any data is returned, meaning a client can never retrieve another client's bookings, messages, or notifications regardless of what query arguments they supply. This server-side enforcement is the authoritative security boundary; any client-side filtering is supplementary.

The Employee group strategy grants elevated permissions only to Cognito-verified group members. Group membership is assigned by an administrator in the Cognito console and cannot be modified by the user or the application. The application reads group membership from the JWT token claims on each login and routes the user accordingly, but no UI element on the client side is the authoritative security gate for sensitive employee operations.

3.8.3 Data Integrity

The atomic double-check booking algorithm described in Section 3.5.4 protects the integrity of the booking schedule by detecting and rejecting conflicting submissions even under concurrent access. The post-check step specifically addresses the race condition that would arise if two clients submitted overlapping bookings within the same time window and both passed the pre-check before either write was committed.

AppSync's optimistic concurrency control (described in Section 3.4.4) protects record integrity for all other mutation types. The combination of these two mechanisms ensures that no booking conflict or data corruption can arise from concurrent access, even in high-traffic scenarios.

3.9 Performance Optimisations

Several targeted optimisations were applied throughout the implementation to ensure a smooth user experience across all supported platforms. Table 3.14 summarises each optimisation and its benefit.

Table 3.14 - Performance Optimisations Applied

Optimisation	Technique	Benefit
Selective UI rebuilds	Data change check before calling <code>setState()</code>	Prevents unnecessary widget tree rebuilds and eliminates flickering

Image caching	<code>cacheWidth</code> parameter on <code>Image.network()</code>	Limits decoded image resolution to display size; reduces GPU memory usage
Text scale clamping	<code>TextScaler.clamp(minScaleFactor: 0.85, maxScaleFactor: 1.20)</code>	Prevents oversized accessibility fonts from breaking fixed-dimension layouts
Portrait lock	<code>SystemChrome.setPreferredOrientations([portrait])</code> on mobile	Prevents landscape reflow issues on narrow studio card grids
Immutable state lists	<code>List.unmodifiable()</code> in all <code>AppState</code> getters	Prevents accidental in-place mutations of shared state from widget code
Pre-signed URL expiry	One-hour expiry per generated S3 URL	Balances security with performance; avoids regenerating URLs on every render
Subscription cleanup	<code>cancel()</code> and null-assignment in <code>_disposeAllSubs()</code>	Prevents memory leaks and duplicate event delivery after sign-out
Chat status caching	Local cache with 5-minute TTL per client email	Reduces GraphQL queries for chat status on the employee clients list
Atomic booking post-check	ID exclusion in conflict re-query	Race-condition protection without locking the database or requiring a transaction

3.10 Summary

This chapter presented the complete design and implementation of the GENZ Studios mobile application. The system was constructed around a cloud-first serverless architecture using the AWS Amplify v2 Dart SDK, with Flutter providing a unified cross-platform presentation layer that targets Android, iOS, and Web from a single codebase. The backend is deployed in the AWS eu-central-1 region and leverages Cognito for authentication, AppSync for a

real-time GraphQL API with owner-based row-level security, DynamoDB as the primary NoSQL database, and S3 for scalable object storage.

Five core data entities were designed with clearly defined access control policies enforced at the API layer. A six-step atomic double-check algorithm eliminates booking conflicts under concurrent access. A hybrid real-time strategy combines GraphQL WebSocket subscriptions for low-latency channels such as chat and studio updates with intelligent polling for notification and booking data where subscriptions are not compatible with the owner-auth model.

The application implements ten functional modules spanning authentication (four dedicated screens plus a session-check splash), onboarding (five guided pages), studio management, booking management, real-time chat and support ticketing, notifications, image storage, AI chatbot integration, hierarchical reporting with PDF export, and user settings with full language and theme switching. Twenty-two screens in total provide each user role with a tailored and appropriately restricted interface.

A four-tier responsive layout system adapts the interface to mobile, tablet, desktop, and large-desktop screen sizes from a single codebase. Full Arabic right-to-left and English left-to-right localisation with more than 200 internationalised strings broadens the application's accessibility. Multiple targeted performance optimisations including selective widget rebuilds, image caching, text scale clamping, and subscription lifecycle management ensure a smooth and resource-efficient experience across all platforms.

The network infrastructure and AI chatbot components of the GENZ Studios project are addressed in their respective chapters by the responsible team members. The following chapter will present the testing methodology and the results of the validation procedures carried out to verify that the mobile application satisfies the requirements established in Chapter Two.